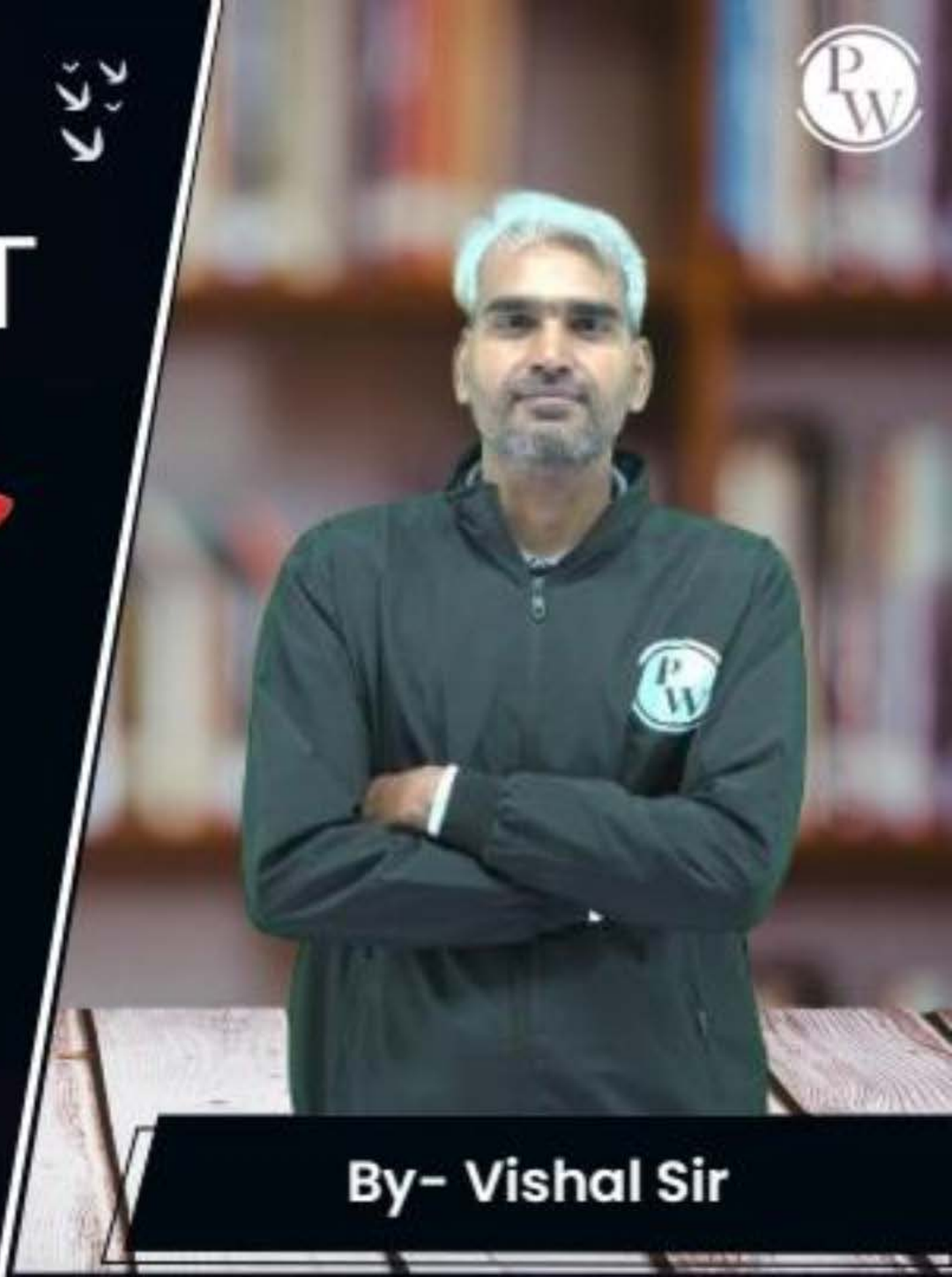
Computer Science & IT



Database Management System

File organization & Indexing

Lecture No. 04



By- Vishal Sir

Recap of Previous Lecture

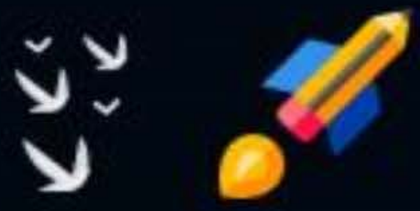


- Topic: Types of index
- Topic: Primary Index { Search key = key (unique) ; File : ordered }
- Topic: Clustering Index { Search key = Non-key (Not unique) ; File :- ordered }
- Topic: Secondary Index { File : Unordered ; Search key :- (key or Non-key) }
- Topic: Multilevel Index

IO Cost = (No. of levels + 1)
{with Multi-level Index}

Slide

Topics to be Covered



Topic B Tree Index



Slide



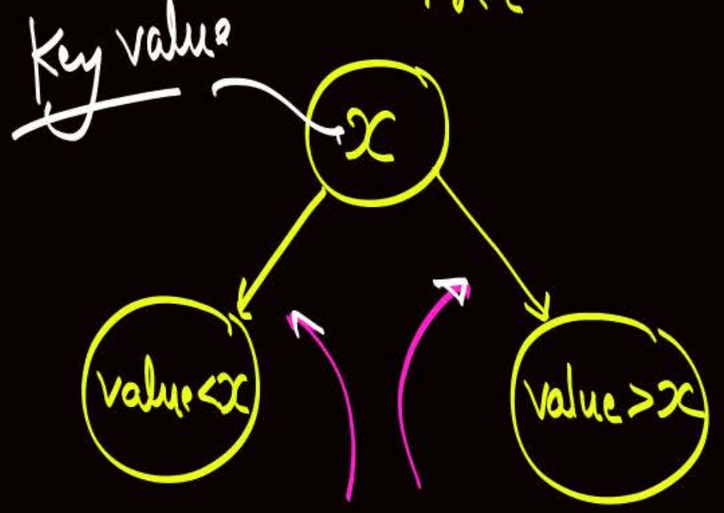
Topic : Dynamic Multi-level Index

Multi-way Search Tree → { B Tree, B+ Tree } } Balanced Search Tree

Balanced Search Tree ↓

All leaf nodes of the tree will be at same level.

Binary Search Tree

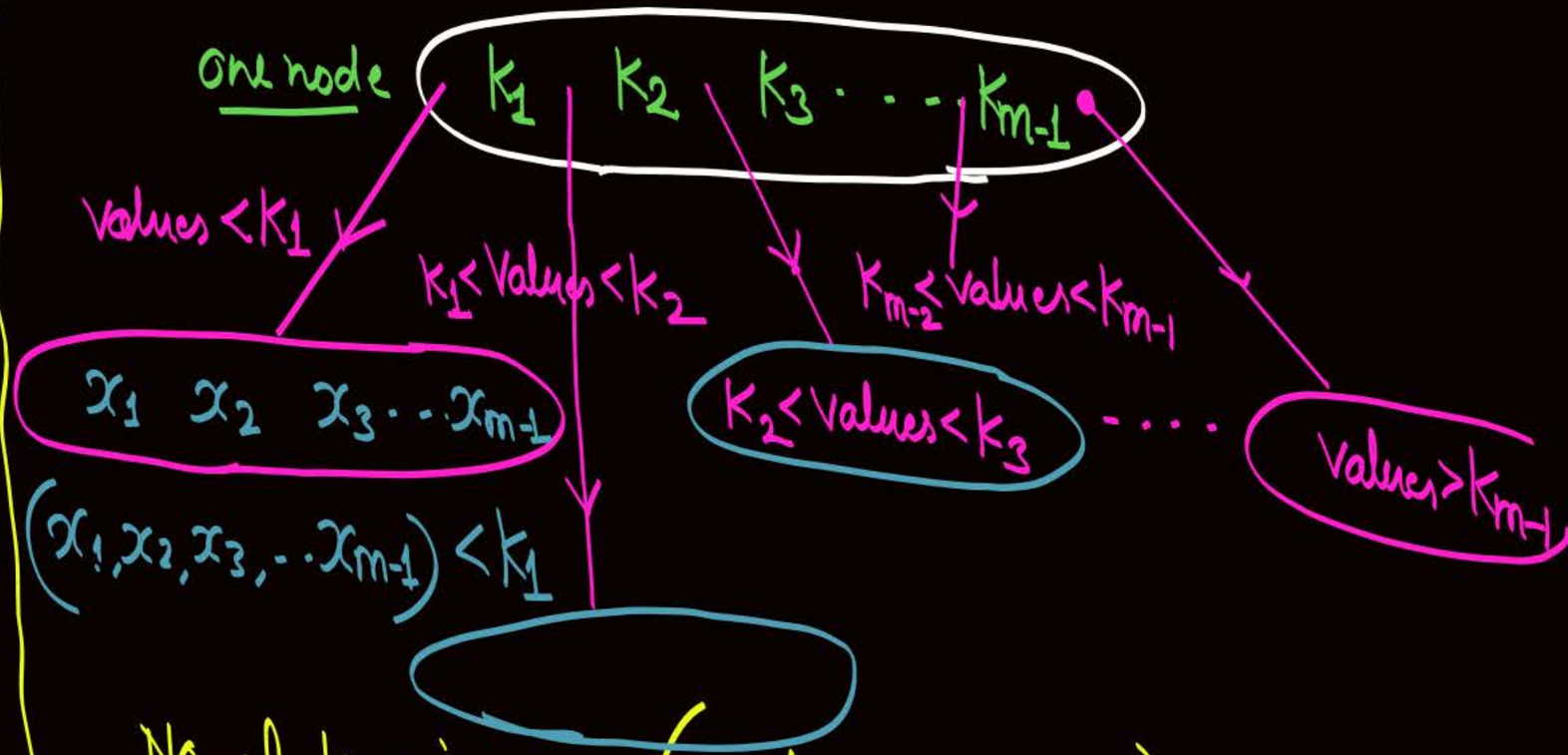


Maximum two children of a node in B.S.T.

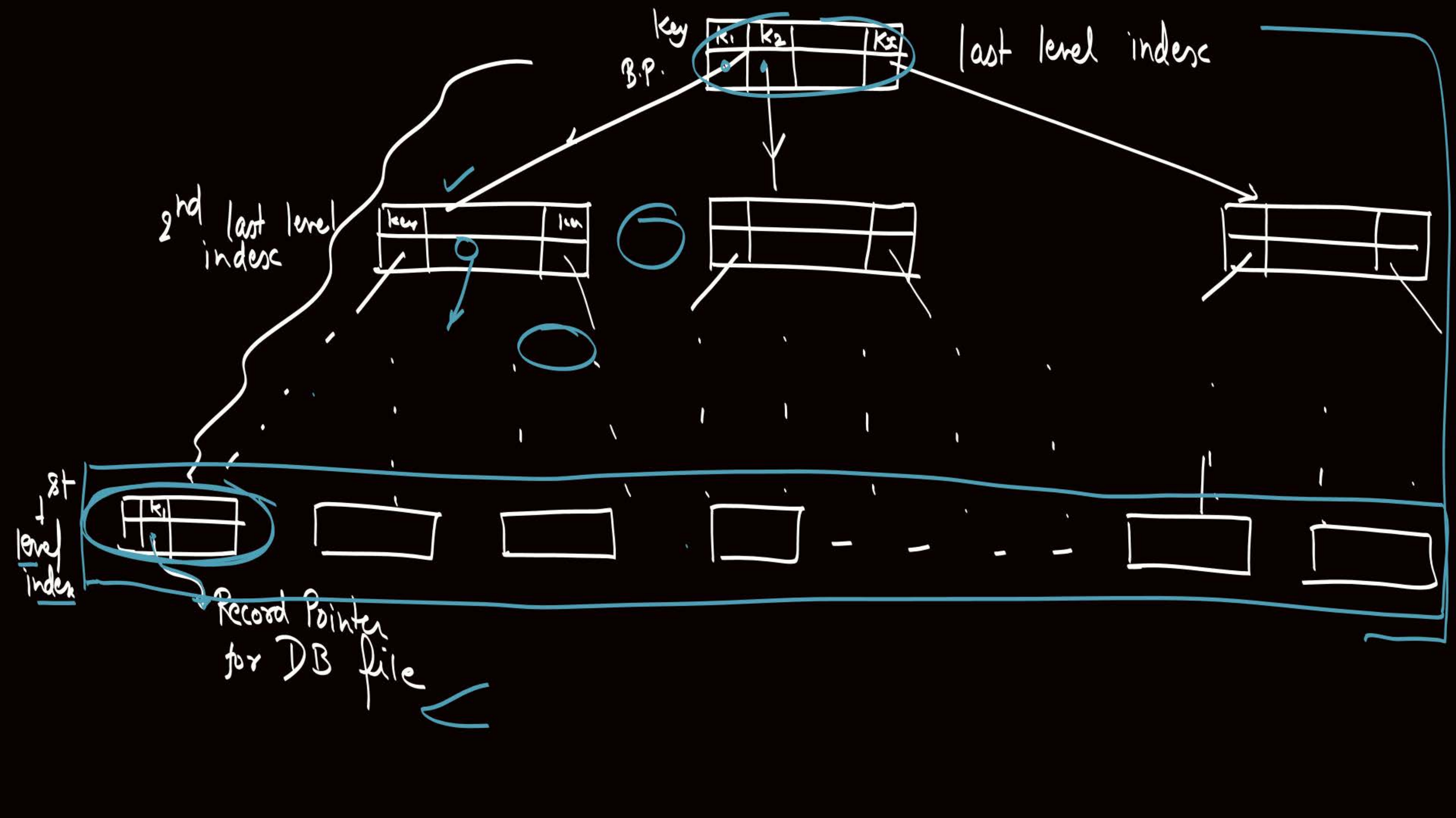
∴ It is a 2-way Search tree.

M-Way Search tree

i.e. Maximum 'm' children of a node



No. of keys in a node = (No. of Child Pointer − 1)


Key
B.P.
K1
K2
K3
last level index
2nd last level index
1st level index
Record Pointer for DB file



Topic : B Tree

* Order of a node of B Tree: Order of a node of B Tree index is defined as maximum number of Child pointer a B Tree node can contain

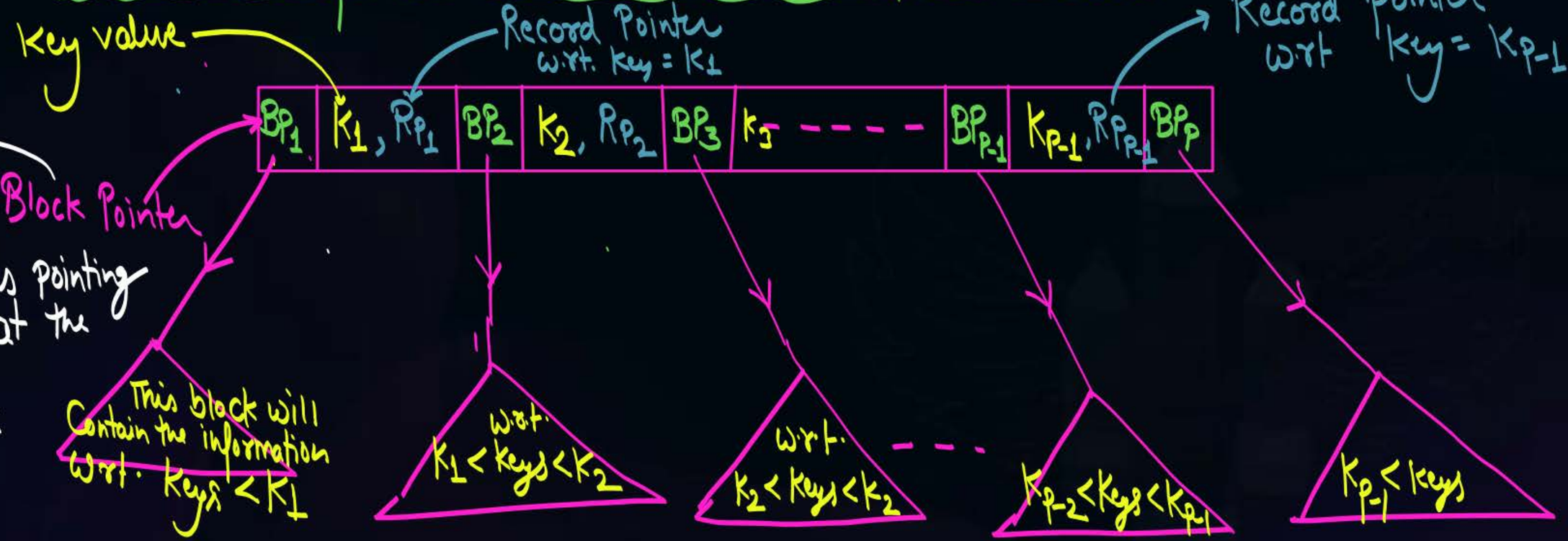


Topic : B Tree

{ i.e. Maximum number of child pointer possible = P
∴ Maximum number of keys possible in a node of B Tree = (P-1)

Let order = P for a node of B Tree index

(1) Structure of an internal node of B Tree :-



Note: In B Tree index, the record pointers corresponding to the keys may be present in the internal node of B Tree as well.

This will not happen w.r.t. Static multi-level index (or) w.r.t. B+ Tree.

In these two cases record pointers will be present only at the last level of Index tree.

* In each node of B Tree along with key values their corresponding record pointer will also be available



Topic : B Tree

* Order = Max. no. of child pointer possible

* Record Pointer (Rp) : Record pointer is the pointer pointing to the database record corresponding to the key value

* Block pointer(Bp) / Tree pointer / Child pointer / Node Pointer :-
  - It is the pointer pointing to the blocks of the index tree at the next level.

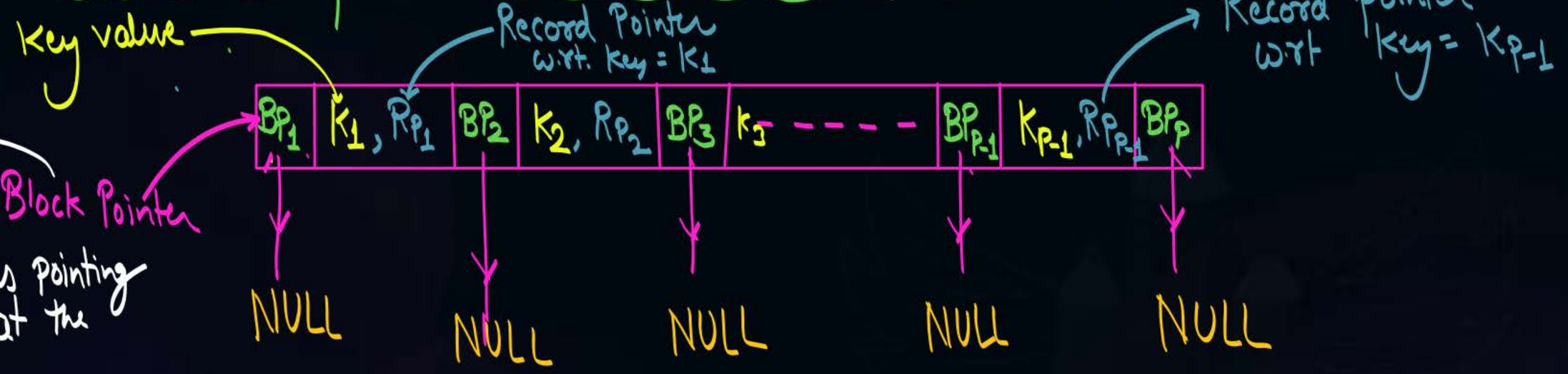


Topic : B Tree

Let order = P { i.e. Maximum number of child pointer possible = P
∴ Maximum number of keys possible in a node of B Tree = (P-1) }

for a node of B Tree index

(2) Structure of a leaf node of B Tree :-





Topic : B Tree

* In a B Tree of order = P,

for both internal as well as leaf node as well as root node

(i) Maximum number of child pointer = P

(ii) Maximum number of keys possible in a B Tree node = (P-1)



Topic : B Tree

* In a B Tree of order = P,

(iii) { Each node { Except root node } will Contain at least $\lceil \frac{P}{2} \rceil$ block pointer and $\left(\lceil \frac{P}{2} \rceil - 1 \right)$ keys

(iv) Root node will Contain at least 2 block pointer and $(2-1) = 1$ key



Topic : B Tree

* In a B Tree of order = P,

(v) All the keys within a node of B Tree must be in ascending order.

(vi) All the leaf nodes must be at the same level



Topic : B Tree

* In a B Tree of order = p

{ where order defines the maximum number of Block Pointer a node can have }

- (i) Max No. of block pointer possible = p
- (ii) Max No of keys possible in a node = p - 1
- (iii) Min No. of block pointer a non-root node must have = $\lceil \frac{p}{2} \rceil$
- (iv) Min No. of keys a non-root node must have = $\lceil \frac{p}{2} \rceil - 1$
- (v) Min No. of block pointer a root node must have = 2
- (vi) Min No. of keys a root node must have = 1



Topic : B Tree

- In a B Tree of order = P

Where Order is defined as maximum number of keys a B Tree node can have

(i) Max No. of block pointer possible = (P+1)

(ii) Max No of keys possible in a node : P

(iii) Min No. of block pointer a non-root node must have : $\lceil \frac{P+1}{2} \rceil$

(iv) Min No. of keys a non-root node must have : $\lceil \frac{P+1}{2} \rceil - 1$

(v) Min No. of block pointer a root node must have = 2

(vi) Min No. of keys a root node must have = 1

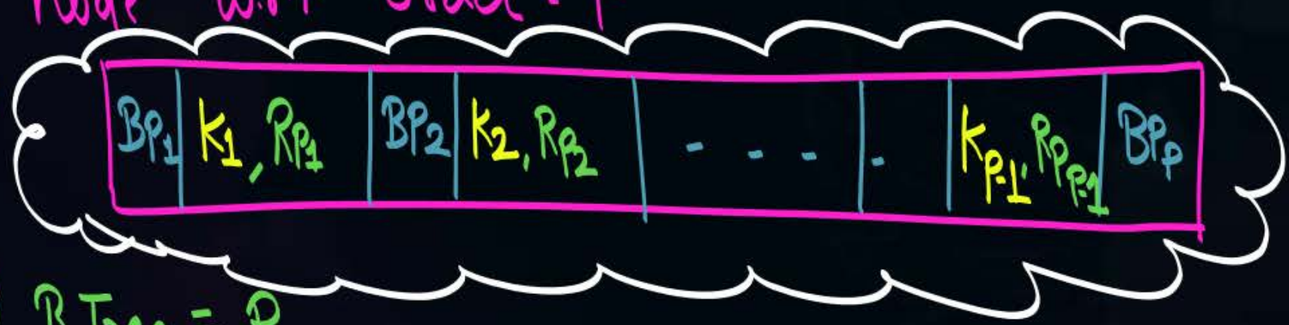


Topic : B Tree

Note: A node of B Tree will be a block of disk

∴ Whatever we want to store in a node of B Tree must fit into a block of disk

∴ B Tree node w.r.t order = P



← it must fit into a single block of disk

i.e. if order of B Tree = P

$$\left[\left(P * \text{Block pointer size}\right) + \underline{P-1} * \left\{\text{Key value Size} + \text{Record pointer Size}\right\}\right] \leq \text{One disk Block size}$$



Topic : B Tree

Consider a relations R with a key filed K. A B-tree of order P is used to access structures on K. Where 'P' denotes the maximum Number of tree pointers in a B-tree index node. Assume that key (k) is 30 bytes long disk block size 1024 byte. Each data (Record) pointer size P_D is 12 bytes long & each block pointer P_B is 10 byte long. In order for each B-Tree Node to fit in a single disk block, the maximum value of P is-

$(P * 10) + P-1 \times (30+12) \leq 1024$

$52P - 42 \leq 1024$

$P \leq \left(\frac{1024+42}{52}\right)$

$P \leq 20.5$

A 19

B 20

C 21

D 22



2 mins Summary

Topic B Tree Index

Slide

THANK - YOU